Agência de Relacionamento

Especificação de Requisitos, Arquitetura e Projeto do Programa

Lab. AED I – Engenharia da Computação – CEFET-MG, Campus Timóteo – Prof. Luciano Moreira – Trabalho 1

Autores: Kauã Ferreira Viana (Parte A – pacote tad) e Luiz Fernando Alves do Prado (Parte B – pacote app)

1. Introdução

1.1 Objetivo

Neste documento apresentamos os requisitos, a arquitetura e o projeto da aplicação *Agência de Relacionamento*, que desenvolvemos em Java. A aplicação lê os cadastros de candidatos de uma agência e identifica, de forma automática, os **potenciais companheiros** de cada um.

1.2 Escopo

A aplicação é de linha de comando e interativa (menu textual). Ela cobre:

- a leitura do arquivo de cadastros candidatos.txt;
- o cálculo dos pares a partir de um grau de interesse mínimo informado pelo usuário;
- seis tarefas de consulta e impressão, escolhidas por menu;
- um programa auxiliar que gera arquivos candidatos.txt aleatórios para testes.

1.3 Definições e siglas

Termo	Significado
TAD	Tipo Abstrato de Dados
Candidato	Pessoa cadastrada na agência
Potencial companheiro	Outro candidato que satisfaz todas as restrições de pareamento em relação a um candidato
Grau de interesse em comum	Soma dos pesos das áreas de interesse que dois candidatos têm em comum
Grau mínimo	Valor informado pelo usuário; o grau em comum precisa ser maior ou igual a ele
Sentinela	Célula inicial da lista que não guarda dado, simplificando inserções e remoções
Tail	Referência para a última célula da lista, que permite inserção no fim em tempo constante

1.4 Restrições impostas pelo enunciado

- A linguagem é Java; nenhuma outra é aceita.
- Os TADs usam **lista simplesmente encadeada com tail e sentinela**.
- É proibido usar estruturas prontas do Java (ArrayList, LinkedList e similares). Todas as estruturas são implementadas pela dupla.
- O programa principal é interativo e a interface pode ser textual.
- É obrigatório um programa auxiliar que gere o candidatos.txt a partir de um dicionário de nomes.
- O trabalho é feito em dupla.

2. Visão geral do sistema

O fluxo de uso é o seguinte:

1. O usuário (ou o gerador) produz o arquivo candidatos.txt.
2. A Main carrega o arquivo por meio do LeitorCandidatos, que devolve uma lista de Candidato.
3. O usuário escolhe, no menu, a tarefa 1 e informa o grau de interesse mínimo. O sistema compara todos os candidatos entre si e preenche a lista de potenciais companheiros de cada um.
4. O usuário consulta os resultados pelas tarefas 2, 3 e 4, ou usa as consultas independentes 5 e 6.

3. Especificação de requisitos

3.1 Requisitos funcionais

Id	Requisito
RF01	O sistema deve ler o arquivo de cadastros (por padrão candidatos.txt, ou o caminho passado como argumento) e criar um Candidato para cada bloco do arquivo. O id do candidato é a sua posição de leitura, de 0 a N-1.
RF02	O sistema deve apresentar um menu interativo, repetido até o usuário escolher sair (opção 0).
RF03 (Tarefa 1)	O sistema deve permitir inserir ou atualizar o grau de interesse mínimo (inteiro maior ou igual a zero) e recalcular os potenciais companheiros de todos os candidatos. Pode ser repetida com outro grau a qualquer momento.
RF04 (Tarefa 2)	Dado o id de um candidato, o sistema deve imprimir o número e os nomes dos seus potenciais companheiros.
RF05 (Tarefa 3)	Dado K, o sistema deve imprimir id e nome dos K candidatos com maior número de potenciais companheiros. Em caso de empate vence o menor id. Se K for maior que o total, imprime todos.

Id	Requisito
RF06 (Tarefa 4)	O sistema deve imprimir id e nome dos candidatos que não têm nenhum potencial companheiro.
RF07 (Tarefa 5)	Dado o código de uma área (1 a 8), o sistema deve imprimir id e nome dos candidatos com interesse nela.
RF08 (Tarefa 6)	Dada a sigla de um estado, o sistema deve imprimir id e nome dos candidatos que residem nele, sem diferenciar maiúsculas de minúsculas.
RF09	O sistema deve avisar o usuário quando as tarefas 2, 3 ou 4 forem pedidas antes de o grau de interesse ter sido definido pela tarefa 1.
RF10	O programa auxiliar deve gerar um candidatos.txt com quantidade de candidatos, arquivo de nomes e arquivo de saída informados por argumentos, ou com valores padrão (30 candidatos, nomes.txt, candidatos.txt).
RF11	O TAD Candidato deve oferecer operações para criar um candidato e atualizar seus campos (nome, estado, sexo, sexo de interesse e áreas).

3.2 Regras de negócio

RN01 – Potenciais companheiros. Dois candidatos distintos u e v são potenciais companheiros quando **as três condições** a seguir são verdadeiras ao mesmo tempo:

- **a)** moram no mesmo estado;
- **b)** a orientação é mútua: o sexo de v atende ao sexo de interesse de u **e** o sexo de u atende ao sexo de interesse de v. O valor I (qualquer sexo) aceita todos;
- **c)** o grau de interesse em comum é maior ou igual ao grau mínimo definido pelo usuário.

Por exigir os dois sentidos, a relação é **simétrica**: se v é companheiro de u, então u é companheiro de v.

RN02 – Grau de interesse em comum. É a soma dos pesos das áreas que os dois candidatos têm em comum:

Código	Área	Peso
1	ESPORTES	1
2	ARTES	2
3	MÚSICA	1
4	CINEMA	1
5	TECNOLOGIA	3
6	ANIMAIS	2

Código	Área	Peso
7	GASTRONOMIA	2
8	CIÊNCIAS	3

RN03 – Dados do cadastro. Estado é a sigla (guardada em maiúsculas); sexo é F ou M; sexo de interesse é F, M ou I; as áreas de interesse são códigos de 1 a 8, sem repetição. Áreas inválidas ou repetidas são ignoradas ao montar o candidato.

RN04 – Desempate. Na tarefa 3, candidatos com o mesmo número de companheiros são ordenados pelo menor id.

3.3 Requisitos não funcionais e restrições

Id	Requisito
RNF01	Implementação em Java (testado com OpenJDK 21), sem bibliotecas externas.
RNF02	Uso exclusivo de estruturas próprias: ListaEncadeada<T> e ListaIdentificadores, ambas com sentinela e tail. Não há ArrayList, LinkedList nem vetores de dados.
RNF03	Interface textual, com validação das entradas numéricas do teclado (texto no lugar de número gera mensagem e nova tentativa).
RNF04	Arquivo ausente, incompleto ou fora do formato deve gerar mensagem de erro clara, sem encerrar o programa com exceção não tratada.
RNF05	Desempenho: com 300 candidatos a tarefa 1 leva menos de 1 segundo; com 2000 leva mais de 20 segundos (ver seção 8.3).
RNF06	Separação em dois pacotes: tad (estruturas e TAD do domínio) e app (leitura, interface e gerador).
RNF07	Código documentado com comentários e testado por classes de teste executáveis por linha de comando.

3.4 Formato do arquivo de entrada candidatos.txt

```
N
****
nome
sigla_estado
sexo
sexo_interesse
numero_areas
area_1
area_2
...
****
(proximo candidato)
```

N é o número de candidatos. Exemplo do arquivo entregue (3 candidatos):

```
3
****
Ana Souza
MG
F
M
3
1
3
5
****
Bruno Lima
MG
M
F
3
3
5
2
****
Carla Dias
SP
F
I
1
2
```

Com grau mínimo 3, Ana e Bruno são companheiros (áreas em comum MÚSICA, peso 1, e TECNOLOGIA, peso 3, somando 4) e Carla fica sem companheiro, pois mora em outro estado.

4. Arquitetura do programa

4.1 Visão em camadas

O sistema é dividido em dois pacotes, com dependência em um único sentido: app usa tad, e tad não conhece app. No pacote app ficam a Main, o GeradorCandidatos e o LeitorCandidatos. No pacote tad ficam o Candidato, a ListaIdentificadores e a ListaEncadeada. O Candidato usa a ListaIdentificadores para guardar as áreas e os companheiros.

4.2 Organização dos arquivos

Arquivo	Pacote	Responsabilidade
ListaEncadeada.java	tad	Lista genérica; guarda os candidatos e os nomes do gerador
ListaIdentificadores.java	tad	Lista de inteiros; guarda as áreas de interesse e os ids dos companheiros
Candidato.java	tad	TAD Candidato
TesteListaEncadeada.java, TesteLista.java, TesteCandidato.java	tad	Testes dos TADs

Arquivo	Pacote	Responsabilidade
LeitorCandidatos.java	app	Leitura e validação do candidatos.txt
Main.java	app	Programa principal: menu e as seis tarefas
GeradorCandidatos.java	app	Programa auxiliar que gera o candidatos.txt
TesteLeitor.java	app	Teste do leitor
nomes.txt	raiz	Dicionário de nomes usado pelo gerador
candidatos.txt	raiz	Arquivo de exemplo

4.3 Decisões de projeto

- **Id como posição.** O id de cada candidato é a posição dele na lista (0 a N-1). Assim, para achar um companheiro basta usar candidatos.get(id). Cada candidato guarda só os ids dos companheiros, não os objetos.
- **Orientação mútua dentro do TAD.** O verificaPotencialCompanheiro olha os dois lados (u atende v e v atende u), por isso a relação já sai simétrica sem precisar tratar nada na Main.
- **Grau mínimo estático.** Deixamos o grau mínimo como campo static do Candidato, porque ele vale para todos os candidatos e só muda pela tarefa 1.
- **Testes mais baratos primeiro.** Na verificação, o cálculo do grau fica por último, para não calcular à toa quando o estado ou a orientação já reprovaram o par.
- **Métodos auxiliares privados.** marcarInteresse, pesoArea, calculaGrauInteresse e atendeOrientacao ficam privados no TAD.

5. Especificação dos TADs

5.1 ListaEncadeada<T>

Lista simplesmente encadeada genérica, com célula sentinela (head), tail e size. A classe interna Celula é privada.

Operação	Descrição
ListaEncadeada()	Cria a lista vazia (sentinela, tail igual ao head, tamanho 0)
void add(T item)	Insere no fim, em tempo constante; itens nulos são ignorados
T get(int i)	Devolve o item da posição i (a partir de 0); lança IndexOutOfBoundsException se i for inválido
int size()	Número de itens, em tempo constante
boolean isEmpty()	Verdadeiro se a lista está vazia
void clear()	Esvazia a lista, reiniciando sentinela, tail e size

Operação	Descrição
String toString()	Texto no formato [a b c]

5.2 ListaIdentificadores

Lista de inteiros com sentinela e tail, usada para as áreas de interesse e para os ids dos potenciais companheiros.

Operação	Descrição
ListaIdentificadores()	Construtor público; cria a lista vazia
void add(int id)	Insere no fim
int get(int i)	Devolve o id da posição i; lança IndexOutOfBoundsException se inválida
int size()	Tamanho atual
boolean isEmpty()	Verdadeiro se vazia
boolean contains(int id)	Verdadeiro se o id existe na lista
void clear()	Esvazia a lista
String toString()	Texto no formato [15 40 8]

5.3 Candidato

Atributos: id, nome, siglaEstado, sexo, sexoInteresse, areas (ListaIdentificadores), potenciaisCompanheiros (ListaIdentificadores, inicialmente vazia) e o campo estático grauMinimo.

Construtor: Candidato(int id, String nome, String siglaEstado, char sexo, char sexoInteresse, ListaIdentificadores areas). Converte estado e sexos para maiúsculas e copia somente as áreas válidas (1 a 8) e sem repetição.

Operações exigidas pelo enunciado

Operação	Descrição
boolean verificaPotencialCompanheiro(Candidato v)	O receptor é o candidato u. Valida em cascata: v não nulo e diferente de u; mesmo estado; orientação mútua; grau em comum maior ou igual ao grau mínimo
void inserePotencialCompanheiro(Candidato v)	Insere o id de v (obtido por getId()) na lista de u; ignora v nulo
int numeroCompanheiros()	Número de potenciais companheiros

Operação	Descrição
ListaIdentificadores listaCompanheiros()	Lista dos ids dos potenciais companheiros
boolean temInteresse(int area)	Verdadeiro se o candidato tem a área na sua lista de interesses
boolean reside(String sigla)	Verdadeiro se mora no estado informado, sem diferenciar maiúsculas

Operações adicionais

Operação	Descrição
void limpaCompanheiros()	Esvazia a lista de companheiros (usada antes de recalcular)
getId, getNome, getEstado, getSexo, getSexoInteresse, getAreas	Acesso aos campos
setNome, setEstado, setSexo, setSexoInteresse, setAreas	Atualização dos campos do cadastro; setAreas copia a lista recebida antes de limpar a atual
static int getGrauMinimo() / static void setGrauMinimo(int)	Acesso ao grau mínimo compartilhado
String toString()	Texto id - nome, usado nas impressões

Operações privadas: marcarInteresse(int) (valida e insere área), pesoArea(int) (peso 1, 2 ou 3 conforme RN02), calculaGrauInteresse(Candidato) (soma os pesos das áreas em comum) e atendeOrientacao(Candidato) (trata I como coringa).

6. Projeto das classes do pacote app

6.1 LeitorCandidatos

O método estático carregarCandidatos(String caminho) usa BufferedReader, lê o N e depois os N blocos (separador ****, nome, estado, sexo, sexo de interesse, número de áreas e as áreas). As áreas são montadas em uma ListaIdentificadores, depois criamos o Candidato com id igual à posição e guardamos na ListaEncadeada<Candidato>. Se der algum erro, o método imprime uma mensagem e devolve null.

6.2 Main

A Main carrega o arquivo (candidatos.txt por padrão, ou o primeiro argumento) e repete o menu até o usuário escolher 0. A leitura de inteiros é feita pelo método lerInteiro, que pede de novo se o usuário digitar texto. A variável paresCalculados impede usar as tarefas 2, 3 e 4 antes da tarefa 1.

Tarefa	Algoritmo
1	Valida grau maior ou igual a zero, chama setGrauMinimo. Para cada candidato u: limpaCompanheiros e laço sobre todos os v, chamando verificaPotencialCompanheiro e inserePotencialCompanheiro
2	Valida o id, imprime numeroCompanheiros e, para cada id da lista de companheiros, busca o nome em candidatos.get(id)
3	Para cada posição de 1 a K, busca o candidato com mais companheiros ainda não impresso. O uso do operador > mantém o menor id no empate. Os ids já impressos ficam em uma ListaIdentificadores, sem vetor
4	Percorre a lista e imprime quem tem numeroCompanheiros() == 0
5	Valida a área (1 a 8) e imprime quem satisfaz temInteresse(area)
6	Imprime quem satisfaz reside(sigla)

6.3 GeradorCandidatos

É o programa auxiliar, usado assim: java app.GeradorCandidatos [quantidade] [arquivo de nomes] [arquivo de saída]. Ele lê o dicionário de nomes (um por linha, ignorando linhas em branco) para uma ListaEncadeada<String> e, para cada candidato, sorteia:

- o nome, a partir do dicionário;
- o estado entre MG, SP e RJ (poucos estados, para facilitar a formação de pares);
- o sexo (F ou M) e o sexo de interesse (F, M ou I);
- de 1 a 8 áreas distintas, usando ListaIdentificadores para evitar repetição.

O arquivo é gravado no formato da seção 3.4. Se os argumentos forem inválidos (quantidade que não é número ou é menor ou igual a zero, arquivo de nomes inexistente ou vazio), o gerador mostra uma mensagem de erro.

7. Tratamento de erros

Situação	Tratamento
Arquivo inexistente	Mensagem de erro de leitura; retorno null; programa termina
Arquivo incompleto	Mensagem "O arquivo está incompleto ou fora do formato esperado."; retorno null
Texto no lugar de número no arquivo	Mensagem de erro de formatação numérica; retorno null
Texto no lugar de número no menu	"Valor invalido, digite um numero inteiro." e nova tentativa

Situação	Tratamento
Opção de menu inválida	"Opção invalida."
Grau negativo	"O grau de interesse não pode ser negativo."
Id, área ou K inválidos	Mensagem específica e retorno ao menu
Tarefas 2, 3 e 4 antes da tarefa 1	"Defina o grau de interesse na tarefa 1 antes."
Posição inválida em get	IndexOutOfBoundsException

8. Análise de desempenho

8.1 Custo das operações

Operação	Custo
add, size, isEmpty, clear	O(1)
get(i), contains, toString	O(n)
calculaGrauInteresse	O(1) na prática (no máximo 8 áreas, cada uma com contains sobre lista de até 8 itens)

8.2 Custo das tarefas

- **Tarefa 1:** N^2 verificações de pares, cada uma com acesso por posição à lista de candidatos. Como get é O(N), o custo total é da ordem de $O(N^3)$.
- **Tarefa 3:** K buscas sobre N candidatos, cada uma com get e contains, o que dá uma ordem de $O(K \cdot N^2)$.
- **Tarefas 4, 5 e 6:** $O(N^2)$ devido ao get dentro do laço.

8.3 Medições e limitação

Com 300 candidatos a tarefa 1 leva menos de 1 segundo; com 2000, mais de 20 segundos. O custo vem do acesso por posição em lista encadeada. Percebemos essa lentidão nos testes com arquivos grandes.

9. Plano de testes e resultados

Todos os resultados obtidos foram iguais aos esperados. Cada teste imprime a saída na tela, e o resultado esperado de cada linha está comentado no código.

Classe de teste	Execução	O que cobre
TesteListaEncadeada	java -cp bin tad.TesteListaEncadeada	Lista vazia, add, toString, get, size, posições inválidas (-1 e 3), clear e novo add

Classe de teste	Execução	O que cobre
TesteLista	java -cp bin tad.TesteLista	add, get em laço, toString, contains, isEmpty, size, posição inválida e clear
TesteCandidato	java -cp bin tad.TesteCandidato	Grau mínimo, simetria, estado diferente, orientação mútua, coringa I, casos limite (nulo e o próprio candidato), área repetida e inválida, setAreas, companheiros, reside, temInteresse e toString
TesteLeitor	java -cp bin app.TesteLeitor candidatos.txt	Arquivo de exemplo, arquivo incompleto, texto no lugar de número e arquivo inexistente

Testes da aplicação (Main e gerador)

- Menu: texto no lugar de número, opção inválida e saída pela opção 0.
- Tarefa 1: grau válido (3), negativo (-5) e texto.
- Tarefa 2 (grau 3): Ana com 1 companheiro (Bruno), Bruno com 1 (Ana), Carla com 0 e id inválido.
- Tarefa 3 (grau 3): K = 1 (Ana, por desempate), K = 2, K = 10 e K = 0.
- Tarefa 4: grau 3 (só Carla) e grau 6 (os três candidatos).
- Tarefas 5 e 6: áreas 3, 2, 8 e 9; siglas mg, SP e rj.
- Gerador: 40 candidatos, conferência do formato e das regras, leitura pelo TesteLeitor e argumentos inválidos.
- Integração: menu completo no arquivo de exemplo e em um arquivo gerado de 40 candidatos, com grau 4, conferido contra um cálculo feito à parte, incluindo a simetria da relação.

10. Instruções de compilação e execução

Na pasta do projeto (a que contém src):

```
javac -d bin src/tad/*.java src/app/*.java
java -cp bin app.GeradorCandidatos 200 nomes.txt candidatos.txt
java -cp bin app.Main candidatos.txt
```

O detalhamento do uso de cada opção está no arquivo LEIAME.TXT. Execute a opção 1 antes das opções 2, 3 e 4.

11. Divisão do trabalho

Membro	Parte	Responsabilidades	Tempo registrado
Kauã Ferreira Viana	A – pacote tad	ListaIdentificadores, ListaEncadeada<T>, Candidato e seus testes; estrutura do projeto e repositório	12h19min (05/10 a 20/10/2026)

Membro	Parte	Responsabilidades	Tempo registrado
Luiz Fernando Alves do Prado	B – pacote app	LeitorCandidatos, Main com menu e tarefas 1 a 6, GeradorCandidatos e seus testes	7h32min (14/10 a 20/10/2026)

O detalhamento diário, por tipo de tarefa, está nos arquivos RELATORIO-kaua.txt e RELATORIO-Luiz.txt.

12. Conclusão

Conseguimos implementar as seis tarefas e dividimos as responsabilidades entre o pacote tad (dados e regras) e o pacote app (entrada, interface e geração de dados). A maior dificuldade que ficou foi o desempenho da tarefa 1 com muitos candidatos, por causa do acesso por posição nas listas encadeadas (seção 8).